

CSC 640 Software Quality

Software Engineering & High-Quality Software
Products (HQSP)

Students will learn to "manage complex software projects" by creating **effective documentation**, planning **feature deliverables**, analyzing **development resources**, assessing **quality metrics**, and enhancing **software artifacts** through "refinements".

We will discuss how we build a **High-Quality Software Product (HQSP)** by managing complexity through **process, design, and automation**.

Write Down Your Answers First

1. What is Software Engineering (SWE)?
2. What is High-Quality Software?
3. What do we actually *do* to build it?
4. If AI can generate code, tests, and documents, what responsibilities remain for software engineers?

The Story in One Line

Software engineering means **managing complexity to deliver useful, maintainable software under time, budget, and team constraints.**

Keep this line in mind; every slide connects to it.

Part 1. The Story

| A Client Asks for One Feature

Our client runs a small web service for a university and says:

"Users need to **log in**. Please ship it in **three weeks**."

It sounds simple: a form, a password check, and a "Welcome!" page.

We will follow this one feature through the whole talk.

| Who Is "the Client"?

The client is anyone whose problem we solve:

- Your boss or a product owner
- Real customers who pay to use our software
- Yourself (when you build utilities for your own work)

These are the people we must make happy.

| Why "Just Login" Is Not Simple

Once we start, questions appear:

- Where are users stored? (a file, a database, a school account?)
- How are passwords stored safely? (hashing, not plain text)
- What happens after 5 wrong passwords? After the user walks away?

- Does the JavaScript front end talk to the PHP back end correctly?
- Who else on the team is changing the same code?

One "simple" feature touches **data, security, UI, API, and teammates.**

| This Is What We Mean by Complexity

We cannot judge complexity by the number of lines of code alone. What matters is:

- **Dependencies:** login depends on the database, the API, and the UI.
- **Interactions:** a change in one part can affect or break another part.
- **Possible states:** logged in, logged out, locked, expired session, ...

Three Main Sources of Difficulty

1. **Complexity** - dependencies, interactions, and states
2. **Unknown unknowns** - things we fail to anticipate
3. **Human factors** - communication and coordination in a team

We are talking about real-world professional software that people pay to use, not toy programs.

| Unknown Unknowns: The Painful Version

Suppose the team works for two weeks **without showing anything** to the client. In week 2, we discover:

- Students must log in with their **school account (single sign-on, SSO)**; only guest users use email/password.
- The hosting server runs an older PHP version.

Even careful teams miss things like this. We just **didn't know** until it was expensive to find out.

Later, we will replay this same project with early feedback.

| Change Exposes Complexity

Unknown unknowns force **change**, and change **exposes and amplifies** complexity that was already there:

- Adding SSO affects the login page, the session code, and user data.
- Complexity also exists *before* any change: in requirements, interactions, and coordination.

Human Factors

- Software is built **by people, for people**.
- A team helps us manage complexity, but it is also a **new source** of complexity: communication, coordination, and misunderstandings.
- Many unknown unknowns come from people: "Oh, I thought *you* were handling guest accounts."

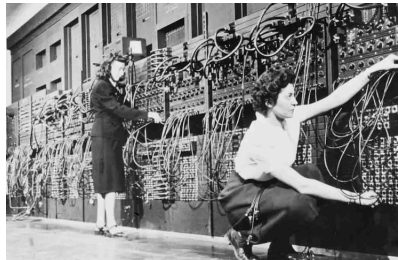
Definition of SWE

- SWE is about the **rules** and **tools** to **build** and **deliver** the software that clients request, **on time, within budget**, in a **team**.
- If SWE is a game, the name of the game is **managing complexity**.
- We cannot conquer complexity once and for all; we manage it **continuously**.

Part 2. What About AI?

| Programming Has Always Changed

Programming was once wiring:



then key punching:



Then searching the web and adapting examples from sites like Stack Overflow:



and now AI can generate code, tests, and documents:



What Has Not Changed

- The goal has never been "making code".
- The goal has always been **solving clients' problems**; code is how we get there.



| AI Is Another Form of Automation

AI can **generate** code, tests, and documents quickly.
But engineers remain responsible for:

- **Understanding** the requirements (Do students need SSO?)
- **Reviewing** the result (Is the password hashed? Is the session secure?)
- **Verifying** quality (Do tests pass? Does it work for real users?)

AI can produce a login page in seconds.

- It cannot promise that the page is **the right one** or **a safe one**.
- The goal of AI to generate code from the prompt, not solving the problem to make client happy.
- Software Engineering has never been about writing code.

Part 3. What Is High Quality?

| Quality Has Many Attributes

The ISO/IEC 25010 quality model includes:

- **Functional suitability** - does it do what users need?
- **Reliability** - does it keep working?
- **Performance efficiency** - is it fast enough?
- **Usability, Security, Compatibility, Portability**
- **Maintainability** - can we change it safely?

Definition of HQSP

A High-Quality Software Product (HQSP) consistently meets **user expectations** while balancing quality attributes such as **reliability, security, maintainability, and efficiency** throughout its lifecycle.

It is a **Product**: clients pay for it, so it must also be delivered **on time** and **within budget**. Quality is a **delivery requirement**, not an optional extra.

| Why Quality Matters (Honestly)

- Users sometimes **tolerate** low quality: low price, urgency, no alternatives, or switching costs.
- But low quality **erodes value over time**, and users leave when a better alternative appears.

So quality does not *guarantee* commercial success, but it supports **lasting value** and **competitiveness**.

| Our Focus: Maintainability

We cannot study every attribute, so this course emphasizes **safe, affordable change**:

- easy to **fix bugs**
- easy to **add features**

Complexity attacks maintainability first, and when maintainability collapses, reliability and security usually follow.

Part 4. How Do We Manage Complexity?

Five Mechanisms

1. **Specify quality early** - agree on what "good" means
2. **Learn early** - check important assumptions cheaply
3. **Process** - manage complexity over **time**
4. **Design** - manage complexity in **structure**
5. **Automation** - let machines do repetitive work

Now, let's **replay the same login project**, this time doing things well.

| 1. Specify Quality from the Start

Quality is not something we check only at the end. We agree on it **first** and check it **continuously**.

Step	Login example
Specified (criterion)	A logged-out session must not access protected resources.
Measured (observation)	On every change, a test sends that request and records whether access was allowed or denied.
Verified (judgment)	The recorded result matches the expected result: access denied.

| 2. Learn Early: Check Assumptions Cheaply

Check the most important assumptions in the **cheapest way possible**. All of these count as early learning:

- **Asking questions:** "Who will log in? Students? Guests?"
- **Checking the environment:** "Which PHP version is on the server?"

- **Small experiments:** "Can we reach the school's SSO test server?"
- **Prototypes:** an ugly login page shown to the client

Research, domain analysis, and threat modeling are also "doing". The point is to **stop overthinking and start checking**, not "always write code first".

| Replay: What Changes?

In the replay, on day 1 we ask questions and check the server. On day 2 we show a rough prototype.

- Client: "No! Students must use their **school account**. Guests keep email/password."
- We learn about SSO and the old PHP version in **week 1**, not week 2.

A cheap check revealed an expensive assumption.

| Then MVPs

- **Prototype**: a quick, low-cost model used to explore an idea and learn before building the real product. It may be incomplete and does not need production-level polish.
- **MVP** (Minimum Viable Product): the smallest product that delivers core value to real users while testing the most important assumptions.

But both have Goals and Limits:

- A prototype or MVP cannot validate everything, including system scale, security, operations, and long-term user behavior.
- MVP means minimum scope, not careless implementation: critical parts such as password storage still require production-level quality.

3. Process: Manage Complexity over Time

Instead of building everything at once, we build **little by little with feedback**.

Week	Deliver	Learn	Revise the plan
1	Guest login (users in a file)	Students need SSO	SSO in week 2
2	Student SSO login	Students stay logged	Session timeout in week 3
3	Session timeout, guest users in MySQL	Tests pass, client accepts	Ship

Under constraints: SSO was not in the original plan, so something had to give. The client agreed to ship essential login first and move **profile editing** to the next release.

Each small delivery reduces the risk from unknown unknowns.

Process in Practice

- **Iterative & Agile** processes: small deliveries, feedback, collaboration, adaptation.
- **CI/CD** (Continuous Integration / Continuous Delivery): automatically build and test every change, so the system stays deployable.

The plan is **not** a promise never to change; it is a shared picture that we **update** as we learn.

4. Design: Manage Complexity in Structure

Break the system into **modules**, and connect them with small **interfaces**. Each module **hides its details** (information hiding, Parnas, 1972).

```
// Illustrative code: class bodies are omitted.  
// The login code depends only on this small interface.  
interface UserStore {  
    public function findByEmail(string $email): ?User;  
}  
  
class FileUserStore implements UserStore { /* JSON file details */ }
```

Why Design Makes Change Safer

In week 3, we moved guest users from a file to MySQL.

- Add `MySQLUserStore` and change only the store-selection line.
- Login still calls `findByEmail()` without knowing SQL.
- New bugs have a clear place to investigate.

```
class MySQLUserStore implements UserStore { /* SQL details */ }
```

Good design does not make change free; it **limits the impact** of change.

- The same separation applies when we add SSO.
- Authentication is different from user lookup, so a separate module and interface limit the impact of future changes.

5. Automation: Let Machines Do Repetitive Work

High-level tools let us **express our intent** instead of every implementation detail, and automation repeats work faster and more reliably than humans.

Tool	Problem it addresses
Git & GitHub	Lost or conflicting changes in a team
Automated tests + GitHub Actions	Bugs that come back after a change
TypeScript	Type errors found only at runtime
Laravel (framework)	Repetitive routing, database, and auth code
Docker, Bash scripts	"It works on my machine" and manual setup steps
AI assistants	Drafting code, tests, and docs (still reviewed by us)

Automations Help, but Do Not Guarantee Quality

A team can use Agile, CI/CD, React, and AI, and still ship insecure, slow, or unusable software.

- Quality must still be **specified** from the start, **measured**, and **verified** throughout development.

Part 5. Rules and Tools

| Rules

Software engineers use **rules** to manage complexity:

- **KISS** (Keep It Simple, Stupid; or politely, Keep It Simple and Straightforward)
- **No Surprises**

- **DRY** (Don't Repeat Yourself)
- **SOLID** (five object-oriented design principles: Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion)

And many more: Buy, don't build; 80/20; ...

KISS: The Simplest Solution That Satisfies the Constraints

- Choose the **simplest solution that still satisfies the constraints** (security, deadline, users' needs).
- Warning: a solution that *looks* simple can **move complexity somewhere else**.

Login example: storing guest passwords in plain text *looks* simple, but it moves the complexity into a future security disaster.

- Using PHP's built-in `password_hash()` is both simple **and** correct.
- Buy, don't build rule to achieve the KISS rule.

| No Surprises: Communicate Early

No Surprises is the #1 **team** rule. It does not mean "never change the plan". It means:

- **Disclose risks early** ("SSO may take longer than we thought.")
- **Communicate changes** as soon as we know them.
- **Update shared expectations** together with the client and the team.

The worst surprise is hearing about a problem on the deadline.

| Tools: A Quick Review

- **Process** manages complexity over time.
- **Design** manages complexity in structure.
- **Automation** handles repetitive work.

Tools keep changing, so professional responsibility means **continued learning**, while the responsibility for quality stays with us.

Our Answers

Compare these with what you wrote at the beginning.

1. **Software Engineering:** managing complexity to deliver useful, maintainable software under time, budget, and team constraints.
2. **High-Quality Software:** software that meets users' needs across many quality attributes; this course emphasizes safe, affordable change (maintainability).

3. **What we do:** specify quality from the start, check assumptions early and cheaply, deliver little by little with feedback (process), isolate responsibilities (design), automate repetitive work, and verify quality continuously.
4. **AI:** another form of automation. It generates artifacts; engineers remain responsible for requirements, review, and quality.

These are first answers; we will refine them throughout the course.

Part 6. CSC 640

| From the Story to Your Semester

The login story showed us the pain: hidden complexity, unknown unknowns, and team coordination.

It also showed us the remedies: specify quality, learn early, iterate, design modules, and automate.

In CSC 640, **you will live this story yourself**: you will build a real product and practice each remedy.

| The Goal

We learn SWE **rules** and **tools** for building and shipping HQSP **on time** and **within budget**.

We also do a **project** to practice them in a realistic setting:

- A PHP-based REST API server with a JavaScript front end.
- If you have your own project, you can do it after discussion and a presentation.

How the Assignments Connect to the Story

Story step	Assignment
Tools ready, story understood	HW1 Make It Ready
Design manages structure	HW2 Software Design
High-level languages express intent	HW3 High-level Programming Language
Deliver a working product	HW4 Project Submission*
Specify, measure, verify quality	HW5 Software Quality Documentation*

 means peer-reviewed evaluation

We check quality in **every** assignment; HW5 collects the evidence into one document.

| The Main Topics

1. Software Process (with intensive testing): Week 2
2. Software Design: Week 3-4
3. High-level programming languages and related tools: Week 5-6

Languages we use:

- Python for Design Patterns
- JavaScript/TypeScript/React for front-end applications
- PHP/MySQL for back-end applications for the project (you can your own language if you choose to do so)

Part 7. Course Orientation

Practical information: prerequisites, setup, grading, and resources.

Prerequisites

CSC 500 Object-Oriented Programming (3 credits) OR some combination of CSC 260 Object-Oriented Programming I, CSC 360 Object-Oriented Programming II, and CSC 364 Data Structures and Algorithms

We assume that students already understand OOP and its principles.

| Setup: Three Main Tools

We use ASE (AI-Enabled Software Engineering) tools and rules extensively.

1. **VSCode** -

<https://code.visualstudio.com/Download>

- Install the **Marp for VSCode** extension

2. **Git/GitHub** - version control and course materials

3. **Markdown** - Marp (presentations) and Hugo (github.io web pages)

Course Materials

Go to the **NKU ASE GitHub repo**:

<https://github.com/nkuase/ASE/tree/main/tools/ASE-onboard/pdf>

- Read the PDFs and download the `.md` files.
- Use the VSCode Marp extension to view them or convert them to **PDF / PPT / HTML**.
- This slide is made using Marp on VSCode.

| WSL2 for Windows Users

All examples and projects assume Linux.

- Mac users can use the same code without modifications.
- Windows users should install WSL2 to run Linux on their PCs.

| Grading

The course structure follows KISS: the simplest structure that meets the learning goals.

- Five assignments (1000 points)
- No exams
- Peer evaluations (two assignments: HW4 and HW5)

| Assignments in Detail

HW1 Make It Ready

- Install CSC 640 tools.
- Understand SWE, the project, and high-quality software.

HW2 Software Design

- Understand OOP, UML, APIEC (core OOP concepts, defined in the HW2 lectures), SOLID, Design Patterns, and Refactorings.

HW3 High-level Programming Language

- Understand JavaScript/Node.js, TypeScript, and React by making simple examples.

HW4 Project Submission

- Build REST API server-side applications first with **raw PHP**, then with **Laravel's framework abstractions**, to see what a framework automates.
- Or choose your own software to build (requires a one-on-one meeting).

HW5 Software Quality Documentation

- Make and publish a Marp/Hugo document about software quality over the semester.

Schedule and Resources

- Schedule, syllabus, and assignments: **Canvas**
- Lectures, code, and questions: GitHub
 - <https://github.com/nkuase/csc640>
 - <https://github.com/nkuase/ASE>

Backup: How Often Do Projects Struggle?

The Standish Group's **CHAOS Report (1994)** surveyed U.S. IT projects:

- 16% completed on time, on budget, with required features
- 53% challenged (late, over budget, or missing features)
- 31% canceled

Scope: old data, one survey, with debated definitions of "success". It does not prove that today's numbers are the same. It shows that delivering software under constraints has long been hard.

Source: The Standish Group, *The CHAOS Report* (1994).